第32课 vector与常用STL

一、课程信息

项目	内容
课次	第32课 / 共40课
主题	vector与常用STL
前置知识	第21课sort函数与STL入门、第31课栈与队列STL
后续衔接	第33课结构体入门（结构体+vector组合使用非常常见）

教学目标（可检验）：

1. 学生能独立使用 `vector<int>` 声明动态数组，并正确使用 `push_back()`、`pop_back()`、`size()`、`v[i]` 完成元素的增删和遍历。
2. 学生能说出 set（不重复集合）和 map（键值对字典）的基本用途，能看懂包含它们的简单代码。
3. 学生能写出万能头文件 `#include<bits/stdc++.h>`，并在竞赛代码中使用。

二、教学重点与难点

重点：
- vector的声明方式：`vector<int> v;`
- vector的四大核心操作：`push_back()`、`pop_back()`、`size()`、`v[i]`
- 用for循环遍历vector
- 万能头文件 `#include<bits/stdc++.h>` 的使用

难点：
- 理解vector与普通数组的区别（动态 vs 固定）
- `size()` 返回的是当前元素个数，不是容量，且类型需注意
- set和map的模板语法（尖括号写法），仅要求了解，不要求熟练使用
- 从"什么都自己写"到"善用STL工具"的思维转变

三、课堂时间分配（共100分钟）

时间	环节	内容	时长
0-8	导入回顾	回顾stack/queue，引出"数组大小不够用怎么办"	8分钟
8-30	新课讲解（上）	vector概念 + 声明 + push_back/pop_back/size/遍历	22分钟
30-45	例题演练（上）	例题1、例题2，边讲边练	15分钟
45-55	课堂练习（上）	练习题1，学生独立完成，教师巡视	10分钟
55-60	休息/答疑	休息，个别答疑	5分钟
60-78	新课讲解（下）	set/map简介 + 万能头文件 + STL工具箱总结	18分钟
78-88	例题演练（下）	例题3、例题4，边讲边练	10分钟
88-96	课堂练习（下）	练习题2、练习题3，学生独立完成	8分钟
96-100	总结预告	总结本课要点 + 布置作业 + 预告下节课	4分钟

> 讲解段：导入8分钟 + 新课上22分钟 + 新课下18分钟 = 最长连续讲解22分钟，符合不超过25分钟的要求。

四、讲稿提纲

4.1 导入回顾（8分钟）——"书架不够大怎么办？"

> 同学们，上节课我们学了stack（栈）和queue（队列），大家还记得吧？stack是"弹夹"，后进先出；queue是"排队"，先进先出。它们都是C++工具箱STL里的好东西。
>
> 今天我们要学STL里另一个**超级常用**的工具——vector。它有多常用呢？可以说在CSP-J比赛里，**几乎每场必用**！
>
> 在学vector之前，老师先问大家一个问题。我们之前用数组的时候，比如 `int a[100]`，这个100是写死的对不对？**如果我不知道要存多少个数怎么办呢？**
>
> 打个比方——普通数组就像家里的**固定书架**。你买的时候就3层，最多放30本书。有一天你的书越来越多，放不下了！怎么办？买新书架？搬家？太麻烦了！
>
> 要是有一种**神奇的书架**——你每放一本书，它就自动多出一格空间，永远不会满，那该多好啊！
>
> 这种"神奇的书架"在C++里就是——**vector**！

（板书：普通数组 = 固定书架，vector = 可以无限加层的书架）

4.2 新课讲解上半场（22分钟）——vector核心操作

第一步：vector怎么创建？（4分钟）

> 要用vector，首先需要引入头文件：

```
#include<vector>
```

> 然后声明一个vector也很简单：

```
vector<int> v;
```

> 我们来看这行代码：
> - `vector` 是类型名，告诉电脑"我要一个动态数组"
> - `<int>` 是尖括号，告诉电脑"里面存整数"——跟第21课学的 `greater<int>()` 那个尖括号是一回事
> - `v` 是你给它起的名字，叫什么都行，叫 `a`、`arr`、`nums` 都可以
>
> 记住：**刚创建出来的vector是空的**，里面一个元素都没有。就像你买了一个全新的、可以伸缩的书架，还没往上面放书。

（板书：`vector<int> v;` 创建一个空的、存int的动态数组）

第二步：往vector里加东西——push_back()（5分钟）

> 书架有了，怎么往里面放书呢？用 `push_back()`：

```
v.push_back(10);   // 放一个10进去
v.push_back(20);   // 再放一个20
v.push_back(30);   // 再放一个30
// 现在v里面是：{10, 20, 30}
```

> `push_back` 的意思是"从**后面**推进去"。还记得上节课stack的 `push` 吗？vector的 `push_back` 多了一个 `_back`，意思是"推到**最后面**"。
>
> 打个比方：你在书架的**最右边**一本一本地放书，先放的在左边，后放的在右边。
>
> 注意写法：是 `v.push_back(10)`，中间有个小圆点 `.`。这个小圆点你可以理解为"**的**"——`v` 的 `push_back`，就是"对v这个书架执行push_back操作"。

（在电脑上演示：每push_back一次，打印一下vector的内容，让学生直观看到变化）

第三步：查看大小和取走最后一个——size()、pop_back()、empty()（5分钟）

> 我想知道书架上现在有几本书？用 `v.size()`：

```
cout << v.size();  // 输出3，因为我们放了3个元素
```

> `size` 就是"大小、数量"的意思。它会告诉你vector里**现在有几个元素**。
> 想把最后一本书取下来？用 `v.pop_back()`：

```
v.pop_back();  // 取走最后一个元素（30被移除了）
// 现在v里面是：{10, 20}
```

> 还记得stack的 `pop` 吗？这里也是一样的意思——弹出去、拿走。`pop_back` 就是"从**最后面**拿走一个"。
> 还有一个好用的：`v.empty()`，判断vector是不是空的。空的返回 `true`，不空返回 `false`。

```
if(v.empty()){
    cout << "书架是空的！";
}
```

（板书汇总表：）

操作	写法	意思	比喻
加元素	`v.push_back(x)`	在最后面加一个x	书架最右边放一本书
删元素	`v.pop_back()`	删掉最后一个	拿走最右边那本书
看大小	`v.size()`	现在有几个元素	书架上有几本书
是否为空	`v.empty()`	是不是空的	书架上有没有书
访问元素	`v[i]`	取第i个元素	拿第i格的那本书

第四步：访问和遍历——v[i] 和for循环（8分钟）

> 我想拿vector里第2个元素怎么办？跟数组一样，用中括号：

```
cout << v[0];  // 输出第1个元素
cout << v[1];  // 输出第2个元素
```

> **跟数组一模一样，下标也是从0开始的！** 这一点大家应该很熟悉了。
> 那怎么把vector里的所有元素都输出来呢？用for循环遍历：

```
for(int i = 0; i < v.size(); i++){
    cout << v[i] << " ";
}
```

> 注意看：循环条件是 `i < v.size()`，不是写一个固定的数字，而是用 `v.size()` 来获取元素个数。因为vector的大小是会变的，所以我们要**动态获取**它的大小。
> 同学们，到这里vector的核心操作就这些了。我们来总结一下vector和普通数组的对比：

（板书对比表）

	普通数组 `int a[100]`	**vector `vector<int> v`**
大小	固定不变	随时可以变大变小
加元素	不能加，满了就溢出	`push_back()` 随便加
知道有几个	自己用变量记	`v.size()` 自动知道
访问方式	`a[i]`	`v[i]`（一样的！）
适合场景	知道最多有多少个	不确定有多少个

> **竞赛建议：** 如果题目告诉你数据最多有多少个，用普通数组就够了；如果不确定，或者需要动态增删，就用vector。后面学图论的时候，vector会特别特别好用！

4.3 新课讲解下半场（18分钟）——set、map简介 + 万能头文件

第五步：认识set——不重复的集合（6分钟）

> 同学们，vector学会了，接下来老师带大家**认识**两个新朋友。注意是**认识**，不是要你们精通，今天只要知道它们能干什么就行。
> 第一个叫 **set**（集合）。
> 打个比方：班级要选课，老师在黑板上写已经报名的同学名字。如果张三已经在上面了，你再写一次"张三"，老师会说"**已经有了，不用重复写**"。
> set就是这样一个"不允许重复"的容器。你往里面丢数字，它会**自动去重**，还会**自动排好序**。

```
#include<set>
using namespace std;

set<int> s;
s.insert(3);  // 放入3
s.insert(1);  // 放入1
s.insert(3);  // 再放3——但是不会重复放！
s.insert(2);  // 放入2
// 此时s里面是：{1, 2, 3}，自动去重、自动排序
cout << s.size(); // 输出3，不是4！
```

> set什么时候有用呢？比如题目问你"一共出现了多少个不同的数"，用set就特别方便——全部塞进去，最后 `s.size()` 就是答案。

第六步：认识map——键值对字典（6分钟）

> 第二个新朋友叫 **map**（映射），你可以把它理解为一本**字典**。
>
> 字典是什么？你查一个**单词**（键），就能找到它的**解释**（值）。map就是这样：给它一个"键"，它对应一个"值"。
>
> 比如我们想记录每个同学的考试分数：

```
#include<map>
using namespace std;

map<string, int> score;
score["小明"] = 95;
score["小红"] = 88;
score["小刚"] = 100;
cout << score["小红"];  // 输出88
```

> 是不是很像查字典？"小红的分数是多少？"——88！
>
> map还有一个常见用法：**统计每个数出现了几次**。

```
map<int, int> cnt;
cnt[5]++;  // 数字5出现了1次
cnt[3]++;  // 数字3出现了1次
cnt[5]++;  // 数字5又出现了1次
cout << cnt[5]; // 输出2，5出现了2次
```

> set和map今天只需要**知道有这么个东西**，以后遇到相关题目我们会再详细讲。

第七步：万能头文件——竞赛神器（3分钟）

> 同学们有没有觉得，学了这么多STL，头文件好多啊？用sort要写 `#include<algorithm>` ，用stack要写 `#include<stack>` ，用vector要写 `#include<vector>` ，用set要写 `#include<set>` ……记不住怎么办？
> 老师教你们一个**竞赛神器**——万能头文件：

```
#include<bits/stdc++.h>
```

> 这一行顶替所有头文件！写了它，你就不需要再写 `<iostream>` 、 `<algorithm>` 、 `<vector>` 、 `<stack>` 、 `<queue>` 、 `<set>` 、 `<map>` ……全都包含了！
> 你可以把它想象成：以前你去上学，要自己一样一样带——铅笔、橡皮、尺子、书本。现在有一个**万能书包**，什么都装好了，背上就走！
> **注意：** 万能头文件只能在竞赛和平时练习中用。正式的软件开发项目中不推荐用，因为它会拖慢编译速度。但在竞赛里，**强烈推荐**！

（板书： `#include<bits/stdc++.h>` 万能头文件，竞赛必备！）

第八步：STL工具箱大总结（3分钟）

> 好，到今天为止，我们学过的STL工具有哪些？一起来数一数：

（板书STL工具箱汇总表）

工具	头文件	用途	比喻	学习课次
`sort`	`<algorithm>`	排序	洗衣机	第21课
`stack`	`<stack>`	后进先出	弹夹/叠盘子	第31课
`queue`	`<queue>`	先进先出	排队	第31课
`vector`	`<vector>`	动态数组	可伸缩书架	本课
`set`	`<set>`	不重复集合	签到表（不重名）	本课（了解）
`map`	`<map>`	键值对	字典	本课（了解）

> 这些工具箱里的东西，都是别人帮我们写好的，直接拿来用。竞赛中**善用STL，事半功倍**！
> 而且以后你们只需要记住一行： `#include<bits/stdc++.h>` ，所有工具全到手！

七、常见错误与应对

错误1：忘记写头文件或写错头文件

```
error: 'vector' was not declared in this scope
```

应对： 最简单的办法——**直接用万能头文件** `#include<bits/stdc++.h>`，一劳永逸。建议学生从今天开始，竞赛代码统一使用万能头文件。

错误2：vector声明时用了方括号

```
vector<int> v[10];   // 错误理解！这是创建了10个vector，不是一个大小为10的vector！
vector<int> v(10);   // 正确！创建大小为10、初始值为0的vector
```

应对： 强调：圆括号 `()` 设置大小，方括号 `[]` 在vector前面是创建"vector数组"（多个vector），含义完全不同。初学者统一用 `vector<int> v;` 加 `push_back()` 来添加元素最安全。

错误3：对空vector使用v[0]访问

```
vector<int> v;
cout << v[0];  // 危险！v是空的，没有第0个元素！
```

应对： 用 `v[i]` 访问之前，确保vector里有足够的元素。可以先用 `v.size()` 检查，或者确保已经 `push_back` 过。空vector直接用下标访问不会报错但结果不可预料（未定义行为），这是一个隐蔽的bug。

错误4：sort对vector排序时写成数组的写法

```
sort(v, v + n);           // 错误！vector不能这样写
sort(v.begin(), v.end()); // 正确！
```

应对： 提醒学生区分：
- 数组排序：`sort(a, a+n)`
- vector排序：`sort(v.begin(), v.end())`

口诀：**"数组用a，vector用begin/end"**。

错误5：`v.size()-1` 在vector为空时溢出

```
vector<int> v;  // 空vector
for(int i = 0; i < v.size() - 1; i++){  // 危险!
    // v.size()返回0，0-1溢出成一个超大的数
}
```

应对： `v.size()` 返回的是无符号整数（`unsigned`），0减1不会变成-1，而是变成一个非常大的数，导致循环出错。解决方法：先判断 `if(v.size() > 0)` 再做减法，或者将 `v.size()` 强制转换为 `int`。

九、下节课预告

> 同学们，今天我们学了vector这个"可伸缩书架"，还认识了set和map两个好帮手，以及竞赛神器——万能头文件。
>
> 下节课，我们要学一个全新的概念——**结构体**。
>
> 什么是结构体呢？想象一下，你的同学有**名字**、**年龄**、**身高**、**成绩**这么多信息。如果用数组来存，你要开好几个数组——名字数组、年龄数组、身高数组……太乱了！
>
> 结构体就像一个"**人物卡片**"——把一个人的所有信息打包在一起，一张卡片就搞定。而且告诉大家，**结构体和vector组合起来**，会变成竞赛中超级好用的搭档！
>
> 回家记得完成洛谷上的两道作业题，下节课见！

> **备课笔记：** 本课是STL系列的第三课（第21课sort入门第31课stack/queue第32课vector/set/map），核心是vector的掌握，set和map仅需了解概念。vector是后续课程（结构体+vector、图的邻接表存储等）的重要基础，务必让学生熟练push_back/size/遍历三件套。万能头文件从今天起建议学生统一使用，减少记忆负担。map遍历的auto语法学生看不太懂是正常的，不必深入解释迭代器，告诉他们"照着写就行，以后会明白"即可。